Lab test

Rules, format, practice specs

Dinu-Ștefan Rusu ADMD

Lab objectives

Prove four skills, once

Write a small app from a one-page spec in one hour: a screen, a Cubit, a network call, and a route.

Worth 30 points

7.5 points per requirement, graded at the end of the test.

Time
1 hour

Submit
To the lab instructor, when the time is up

Rules

1. You work alone. No pair programming, no shared screens, no copying a classmate's code.
2. One hour, one sitting, starting when you receive the spec.
3. You must be able to explain any line the grader points to, in your own words.
4. No new packages beyond what Labs 2 to 6 already added, unless the spec names one.

The clock does not pause, and the hour is not extended.

Test format

One page, handed at the start

An app idea and four requirements. Nothing changes once the clock starts.

One hour, then you submit

Work until time is up, then submit your solution to the lab instructor. No extensions.

Every spec asks for the same four kinds of requirement, described next.

Requirement types

A screen

Built from the layout widgets: Row, Column, ListView, Card. No hardcoded pixel offsets.

State in a Cubit

An Equatable state class, `emit`, and a BlocBuilder. Not a raw setState.

One network call

A single HTTP call parsed into a model class with fromJson. Not a hardcoded map.

One navigation

A go_router route that carries data and pushes to a second screen.

Rubric

Requirement	Full credit (7.5 p)	Partial credit (3.75 p)
Screen	Matches the spec's widgets, no overflow	Screen exists, but the layout differs from spec
Cubit state	UI updates through emit, never setState	Cubit exists, but a screen still calls setState
Network + model	Parses the real response into a model	Call is made, but fields come from a raw map
go_router nav	Navigates and returns with the right data	Route exists, but push or data is wrong

Anything not attempted scores zero.

Submission

1. Stop working when the hour ends.
2. Submit your solution to the lab instructor.
3. Each requirement is graded in the order of the spec.
4. Be ready to explain any line of your code.

Watch out

What you submit when time is up is what is graded. The test cannot be taken later.

Preparation

Requirement	Taught in
Screen from the layout widgets	Lab 3
State in a Cubit	Lab 5
Network call and a model class	Lab 4
Navigation with go_router	Lab 6

Every requirement was covered in Labs 3 to 6. Re-read the Task slides and the reference code in those four labs. Redo one small screen from memory before you come.

Practice A: habit tracker

An app that lists daily habits the user can check off.

1. A `ListView.builder` of habit cards, each with a checkbox, and a button that opens a dialog to add one.
2. A `HabitsCubit` with an Equatable state holding the habit list. Toggling or adding emits a new state.
3. On start, fetch `/users/1` from JSONPlaceholder and parse it into a `User` model shown in the AppBar.
4. Routes `/` and `/habit/:id`. Tapping a card pushes to the detail route.

Grader checks

- [ ] Each piece sits in its layer: widget, Cubit, repository.
- [ ] A checkbox toggle goes through the Cubit, never setState.
- [ ] The AppBar name comes from the API, not a hardcoded string.
- [ ] The back button returns from the detail screen to the list.

Practice B: campus events board

An app that lists campus events, with filtering and a detail view.

1. A row of filter chips (All, Today) above a scrollable list of event cards.
2. An `EventsCubit` with a state holding the event list and the selected filter. Picking a chip emits a filtered state.
3. On start, fetch `/posts` from JSONPlaceholder and parse each item into an `Event` model.
4. Routes `/` and `/event/:id`. Tapping a card pushes to a detail screen with the full text.

Grader checks

- [ ] A chip filters the list through the Cubit, not a local variable.
- [ ] Every field on screen comes from the Event model, never a raw map key.
- [ ] The detail route reads the id from the URL.
- [ ] A slow network call shows a loading state.

Common mistakes

`Bad state: field does not exist for the class`	The JSON keys do not match the model. Print the raw response before writing fromJson.
`No routes matched location "/habit/3"`	The path pushed is not in the go_router route list, or a name is misspelled.
`A blank screen, nothing in the console`	The Cubit never emitted after the network call finished. Check that the future completes.
`setState() or emit() called after dispose()`	A response arrived after the screen closed. Check `mounted` before calling either.

Submission

Each requirement graded · submitted when time is up